

Lecture 08 : TCP and UDP (Part I)

Introduction

In the vast landscape of the internet, data travels through multiple layers, each with a specific role. This lecture delves into the critical transport layer, focusing on two fundamental protocols: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol). Understanding these protocols is paramount for anyone seeking to grasp how applications communicate reliably or efficiently across networks. We will explore their distinct functionalities, how they interact with the underlying IP layer, and the crucial concept of port numbers that enables precise process-to-process communication on a single machine. This foundational knowledge is essential for comprehending network security, application development, and overall internet architecture.

Core Concepts

Term	Definition	Significance
TCP	Transmission Control Protocol; a connection-oriented, reliable, full-duplex, byte-stream service protocol.	Ensures data integrity, order, and flow control for applications requiring high reliability.
UDP	User Datagram Protocol; a connectionless, unreliable datagram service protocol.	Offers speed and efficiency for applications where occasional data loss is acceptable, like streaming.
Port Number	A 16-bit identifier assigned to a specific user process or application running on a computer.	Enables the operating system to direct incoming network packets to the correct application.
Connection-Oriented	A communication model where a logical connection is established, maintained, and terminated between endpoints.	Guarantees ordered and reliable delivery of data, as seen in TCP.

Term	Definition	Significance
Connectionless	A communication model where each packet is treated independently without prior connection setup.	Provides faster, lower-overhead communication but without reliability guarantees, as in UDP.
Reliability	The assurance that data transmitted will arrive at the destination without errors, loss, or duplication, and in the correct order.	A key feature of TCP, achieved through mechanisms like acknowledgements and retransmissions.
Full-Duplex	A communication mode allowing simultaneous two-way data transmission between two points.	Both sender and receiver can transmit and receive data concurrently, a characteristic of TCP.
Byte-Stream Service	A service where data is treated as an ordered sequence of bytes, with both ends tracking byte numbers.	Allows TCP to manage data flow and reassembly precisely, ensuring data integrity.
Checksum	A small block of data derived from a larger block of data, used to detect errors during transmission.	Both TCP and UDP use checksums to verify the integrity of received packets.
Positive Acknowledgement	A mechanism where the receiver explicitly confirms successful receipt of data to the sender.	A core TCP reliability feature, informing the sender which bytes have been received.
Timeout	A predefined interval after which a sender retransmits data if no acknowledgement is received.	TCP's mechanism to handle lost packets or acknowledgements by re-sending data.
Flow Control	A mechanism to manage the rate of data transmission between sender and receiver	TCP dynamically adjusts transmission speed to

Term	Definition	Significance
	to prevent overwhelming the receiver.	match the receiver's capacity.
Sequencing	The process of reordering received data packets into their original transmission order.	TCP's responsibility to deliver data to the application in the correct sequence, despite IP's arbitrary delivery.
Ephemeral Port Number	A temporary port number assigned by the operating system to a client process initiating a connection.	Used for the duration of a communication session and then discarded, typically for client-side applications.
Well-Known Port Number	Standardized port numbers (1-1023, extended to 4095) reserved for specific common services.	Allows clients to easily locate and connect to standard services like HTTP (80) or SMTP (25).

Detailed Analysis

1. TCP vs. UDP in the Transport Layer

At the transport layer of the TCP/IP protocol suite, two primary protocols facilitate communication between applications: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol). User programs interact with either TCP or UDP to send or receive data across the internet. For instance, if a user program wants to send data, it can pass it to TCP, which then hands it to the IP layer for routing. Both TCP and UDP operate above the IP layer, which is inherently an unreliable, datagram-based service. This means that IP datagrams can be lost, duplicated, or arrive out of order. The choice between TCP and UDP depends entirely on the application's requirements for reliability and speed.

2. TCP's Core Features and Mechanisms

TCP is designed for applications that demand high reliability and ordered data delivery. Its key characteristics include:

- **Connection-Oriented Service:** Before data transfer, TCP establishes a logical connection between the sender and receiver, similar to a virtual

circuit. This connection is maintained throughout the communication and then terminated. While the TCP layer provides this connection-oriented illusion, the underlying IP layer still sends individual datagrams that may take different paths and arrive out of order. TCP works hard to reassemble and reorder these datagrams to present a continuous, ordered stream to the application.

- **Reliable Data Transfer:** TCP ensures that data arrives at the destination correctly and completely. This reliability is achieved through several mechanisms:
- **Checksum:** Each TCP segment includes a checksum to detect corruption during transmission. If a checksum error is found, the segment is discarded.
- **Positive Acknowledgement:** The receiver sends acknowledgements (ACKs) to the sender, indicating which bytes have been successfully received. For example, an ACK for byte 1050 means all bytes up to 1050 have been received.
- **Timeouts:** If the sender does not receive an acknowledgement for a transmitted segment within a defined time interval, it assumes the segment (or its ACK) was lost and retransmits the data.
- **End-to-End Flow Control:** TCP dynamically adjusts the data transmission rate to prevent the sender from overwhelming the receiver. This ensures that the receiver can process data at its own pace.
- **Sequencing:** TCP assigns sequence numbers to bytes of data. Even if IP delivers packets out of order, TCP reassembles them into the correct sequence before delivering them to the application.
- **Full-Duplex Communication:** TCP allows simultaneous two-way communication, meaning both parties can send and receive data concurrently.
- **Byte-Stream Service:** TCP treats data as an unbroken stream of bytes. Both the sender and receiver keep track of byte numbers, enabling precise management of how much data has been sent, received, and is yet to be transmitted.

3. UDP's Characteristics

In contrast to TCP, UDP is a simpler, connectionless protocol that prioritizes speed over reliability. It is often described as being very similar to IP, with minimal additional features:

- **Connectionless Service:** UDP does not establish a connection before sending data. Each UDP packet (datagram) is sent independently, without any prior handshake or connection state.
- **Unreliable Datagram Service:** Like IP, UDP does not guarantee delivery, order, or protection against duplication. If a UDP datagram is lost or corrupted, UDP does not attempt to retransmit it.
- **Minimal Overhead:** Due to its simplicity, UDP has less overhead than TCP, making it faster for applications where latency is critical and some data loss is acceptable (e.g., live video streaming, online gaming, DNS queries).
- **Additional Features over IP:** UDP adds two main features that IP lacks at the transport layer:
- **Checksum:** UDP includes a checksum to detect corruption in the UDP datagram. If an error is detected, the datagram is typically discarded.
- **Port Numbers:** UDP incorporates port numbers to identify specific user processes on a host, allowing multiple applications to use UDP simultaneously.

4. The Role of Port Numbers

Port numbers are crucial for enabling process-to-process communication on a single computer. When a packet arrives at a computer, the IP address identifies the machine, but the port number specifies which particular application or process on that machine should receive the data.

- **Unique Process Identification:** Every running process on a computer that communicates over the network is assigned a unique 16-bit port number.
- **Source and Destination Ports:** When a process sends a message, the packet header includes a source port number (identifying the sending process) and a destination port number (identifying the intended receiving process on the remote host).
- **Packet Delivery:** Upon receiving a packet, the transport layer uses the destination port number to deliver the packet to the correct application. For example, if Process 1 (port 10) sends data, and a response comes back with destination port 10, the computer knows to deliver it to Process 1.

- **Quantity:** Port numbers are 16-bit quantities, allowing for 2^{16} (approximately 65,536) unique port numbers.

5. Port Number Categories and the `services` File

Port numbers are categorized to manage their usage effectively:

- **Well-Known Port Numbers:** These are standardized port numbers, originally from 1 to 1023, now extended up to 4095, reserved for common network services. Examples include:
 - SMTP (Mail): Port 25
 - HTTP (Web): Port 80
 - FTP (File Transfer): Control (Port 21), Data (Port 20)
 - Telnet (Remote Login): Port 23
 - DNS (Name Server): Port 53 (both TCP/UDP)
 - Echo: Port 7 (both TCP/UDP)
 - Daytime: Port 13 (both TCP/UDP)
 - TFTP (Trivial FTP): Port 69 (UDP)
- **Ephemeral Port Numbers:** These are temporary port numbers assigned dynamically by the operating system to client processes when they initiate a connection. They are typically chosen from the range above 4095 and are discarded once the communication session ends. This allows clients to receive responses from servers without needing a pre-assigned, well-known port.

Information about well-known services and their corresponding port numbers is stored in a `services` file on most operating systems:

* **Unix/Linux:** `/etc/services`

* **Windows:** `C:\Windows\System32\drivers\etc\services`

This text file lists service names, their port numbers, the protocol (TCP or UDP), and optional comments or aliases.

6. Establishing a TCP Connection (Association)

For TCP, establishing a connection requires specifying several pieces of information, collectively referred to as an "association." This five-tuple uniquely identifies a TCP connection:

1. **Protocol:** Always TCP for a TCP connection.
2. **Local IP Address:** The IP address of the local machine.
3. **Remote IP Address:** The IP address of the remote host.

4. **Local Port Number:** The port number used by the local process (often an ephemeral port for clients).
5. **Remote Port Number:** The port number used by the remote process (often a well-known port for servers).

When a client initiates communication with a server, it must know the server's IP address and the well-known port number of the service it wishes to access. This information allows the client to form the destination part of the association, while the local IP and an assigned ephemeral port complete the source part. This association is automatically managed by the underlying networking software.

Key Takeaways

- [] TCP and UDP are the primary transport layer protocols, offering different levels of reliability and speed.
- [] TCP provides a connection-oriented, reliable, full-duplex, byte-stream service, ensuring data integrity and order.
- [] TCP achieves reliability through checksums, positive acknowledgements, timeouts, flow control, and sequencing.
- [] UDP provides a connectionless, unreliable datagram service, prioritizing speed and efficiency.
- [] Both TCP and UDP utilize port numbers to direct incoming data to specific applications or processes on a host.
- [] Port numbers are 16-bit identifiers, categorized into well-known ports (for standard services) and ephemeral ports (for temporary client connections).
- [] The `services` file on operating systems stores mappings between service names, port numbers, and protocols.
- [] A TCP connection is uniquely identified by an "association," a five-tuple comprising protocol, local/remote IP addresses, and local/remote port numbers.

Conclusion

TCP and UDP form the bedrock of application-level communication over the internet, each serving distinct purposes based on the reliability and performance needs of different applications. TCP's robust, connection-oriented nature ensures that critical data, like emails or web pages, arrives intact and in order, despite the inherent unreliability of the underlying IP layer. Conversely, UDP's lightweight, connectionless approach is ideal for real-time applications where speed is paramount and occasional data loss is tolerable. The concept of port numbers is fundamental to both, enabling multiple applications to coexist and communicate effectively on a

single machine. This lecture has laid the groundwork for understanding how these protocols function; future discussions will delve deeper into their header formats and the intricate processes of TCP connection establishment and termination.